

如何生成分布式雪花算法ID

分布式系统中，有一些需要使用全局唯一 ID 的场景，这种时候为了防止 ID 冲突可以使用 36 位的 UUID，但是 UUID 有一些缺点，首先他相对比较长，另外 UUID 一般是无序的。

有些时候我们希望能使用一种简单些的 ID，并且希望 ID 能够按照时间有序生成。

什么是雪花算法

Snowflake 中文的意思是雪花，所以常被称为雪花算法，是 Twitter 开源的分布式 ID 生成算法。

Twitter 雪花算法生成后是一个 64bit 的 long 型的数值，组成部分引入了时间戳，基本保持了自增。

SnowFlake 算法的优点：

1. 高性能高可用：生成时不依赖于数据库，完全在内存中生成。
2. 高吞吐：每秒钟能生成数百万的自增 ID。
3. ID 自增：存入数据库中，索引效率高。

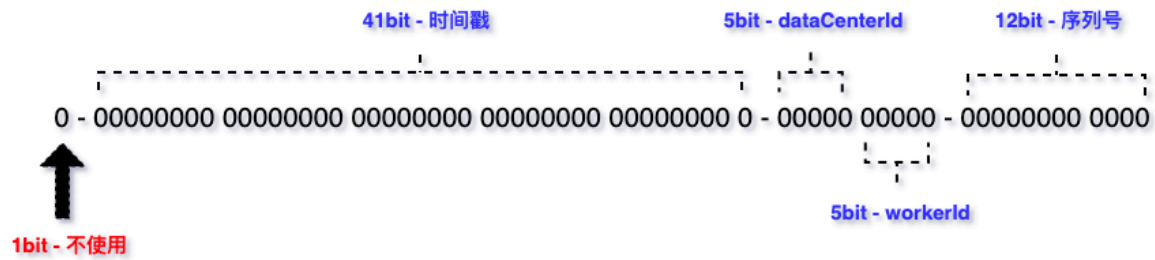
SnowFlake 算法的缺点：

依赖与系统时间的一致性，如果系统时间被回调，或者改变，可能会造成 ID 冲突或者重复。

1. 雪花算法组成

snowflake 结构如下图所示：

snowflake - 64bit

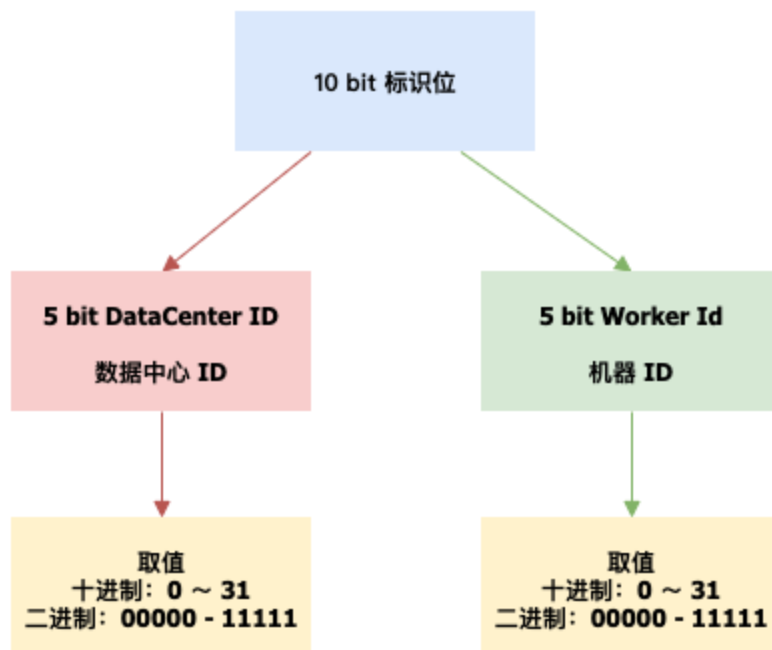


包含四个组成部分：

不使用： 1bit，最高位是符号位，0 表示正，1 表示负，固定为 0。

时间戳： 41bit，毫秒级的时间戳（41 位的长度可以使用 69 年）。

标识位： 5bit 数据中心 ID，5bit 工作机器 ID，两个标识位组合起来最多可以支持部署 1024 个节点。



序列号：12bit 递增序列号，表示节点毫秒内生成重复，通过序列号表示唯一，12bit 每毫秒可产生 4096 个 ID。

通过序列号 1 毫秒可以产生 4096 个不重复 ID，则 1 秒可以生成 $4096 \times 1000 = 4096000$ ID。

默认的雪花算法是 64 bit，具体的长度可以自行配置。如果希望运行更久，**增加时间戳的位数**；如果需要支持更多节点部署，**增加标识位长度**；如果并发很高，**增加序列号位数**。

总结：雪花算法并不是一成不变的，可以根据系统内具体场景进行定制。

2. 雪花算法适用场景

因为雪花算法有序自增，保障了 MySQL 中 B+ Tree 索引结构插入高性能。

所以，日常业务使用中，雪花算法更多是被应用在数据库的主键 ID 和业务关联主键。

雪花算法生成 ID 重复问题

假设：一个订单微服务，通过雪花算法生成 ID，共部署三个节点，标识位一致。

此时有 200 并发，均匀散布三个节点，三个节点同一毫秒同一序列号下生成 ID，那么就会产生重复 ID。

通过上述假设场景，可以知道雪花算法生成 ID 冲突存在一定的前提条件：

1. 服务通过集群的方式部署，其中部分机器标识位一致。
2. 业务存在一定的并发量，没有并发量无法触发重复问题。
3. 生成 ID 的时机：同一毫秒下的序列号一致。

1. 标识位如何定义

如果能保证标识位不重复，那么雪花 ID 也不会重复。

通过上面的案例，知道了 ID 重复的必要条件。如果要避免服务内产生重复的 ID，那么就需要从标识位上动文章。

我们先看看开源框架中使用雪花算法，如何定义标识位。

Mybatis-Plus v3.4.2 雪花算法实现类 Sequence，提供了两种构造方法：无参构造，自动生成 dataCenterId 和 workerId；有参构造，创建 Sequence 时明确指定标识位。

Hutool v5.7.9 参照了 Mybatis-Plus dataCenterId 和 workerId 生成方案，提供了默认实现

一起看下 Sequence 的创建默认无参构造，如何生成 dataCenterId 和 workerId。

```
public static long getDataCenterId(long maxDatacenterId) {
    long id = 1L;
    final byte[] mac = NetUtil.getLocalHardwareAddress();
    if (null != mac) {
        id = ((0x000000FF & (long) mac[mac.length - 2])
            | (0x0000FF00 & (((long) mac[mac.length - 1]) << 8))) >> 6;
        id = id % (maxDatacenterId + 1);
    }

    return id;
}
```

入参 maxDatacenterId 是一个固定值，代表数据中心 ID 最大值，默认值 31。

为什么最大值要是 31？因为 5bit 的二进制最大是 11111，对应十进制数值 31。

获取 dataCenterId 时存在两种情况，一种是网络接口为空，默认取 1L；另一种不为空，通过 Mac 地址获取 dataCenterId。

可以得知，dataCenterId 的取值与 Mac 地址有关。

接下来再看看 workerId。

```
public static long getWorkerId(long datacenterId, long maxWorkerId) {
    final StringBuilder mpid = new StringBuilder();
    mpid.append(datacenterId);
    try {
        mpid.append(RuntimeUtil.getPid());
    } catch (UtilException ignore) {
        //ignore
    }
    return (mpid.toString().hashCode() & 0xffff) % (maxWorkerId + 1);
}
```

入参 maxWorkderId 也是一个固定值，代表工作机器 ID 最大值，默认值 31；datacenterId 取自上述的 getDatacenterId 方法。

name 变量值为 PID@IP，所以 name 需要根据 @ 分割并获取下标 0，得到 PID。

通过 MAC + PID 的 hashCode 获取 16 个低位，进行运算，最终得到 workerId。

2. 分配标识位

Mybatis-Plus 标识位的获取依赖 Mac 地址和进程 PID，虽然能做到尽量不重复，但仍有小几率。

标识位如何定义才能不重复？有两种方案：**预分配和动态分配**。

预分配

应用上线前，统计当前服务的节点数，人工去申请标识位。

这种方案，没有代码开发量，在服务节点固定或者项目少可以使用，但是解决不了服务节点动态扩容性问题。

动态分配

通过将标识位存放在 Redis、Zookeeper、MySQL 等中间件，在服务启动的时候去请求标识位，请求后标识位更新为下一个可用的。

通过存放标识位，延伸出一个问题：雪花算法的 ID 是 **服务内唯一还是全局唯一**。

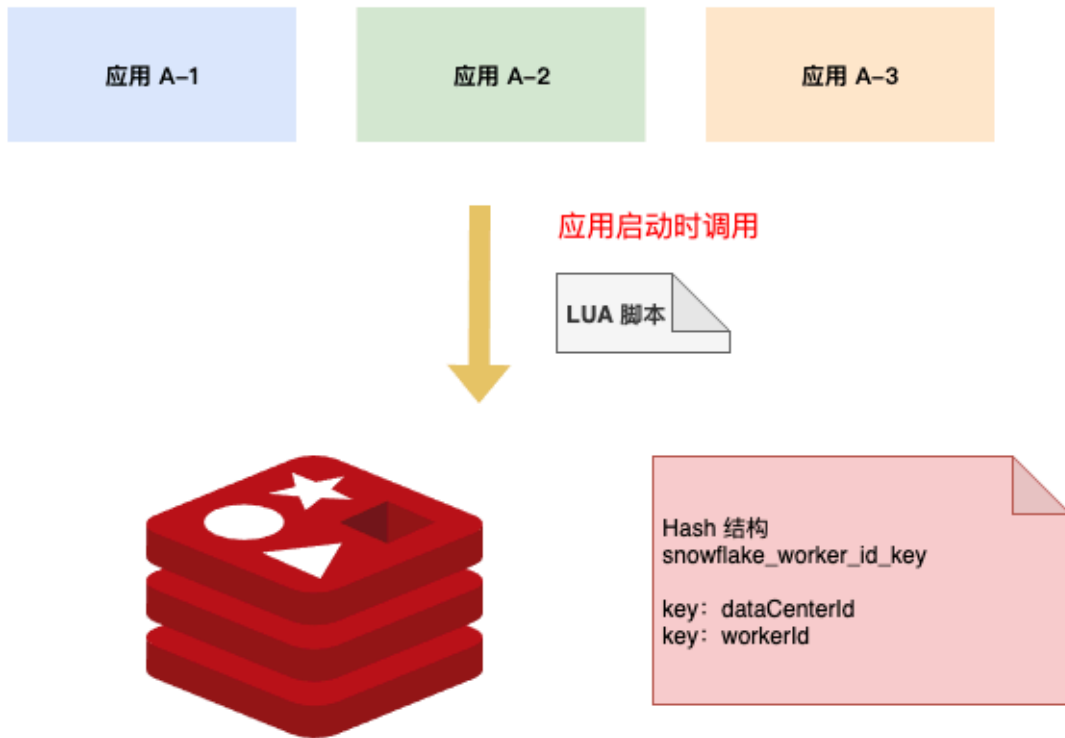
以 Redis 举例，如果要做服务内唯一，存放标识位的 Redis 节点使用自己项目内的就可以；如果是全局唯一，所有使用雪花算法的应用，要用同一个 Redis 节点。

两者的区别仅是 **不同的服务间是否公用 Redis**。如果没有全局唯一的需求，最好使 ID 服务内唯一，因为这样可以避免单点问题。

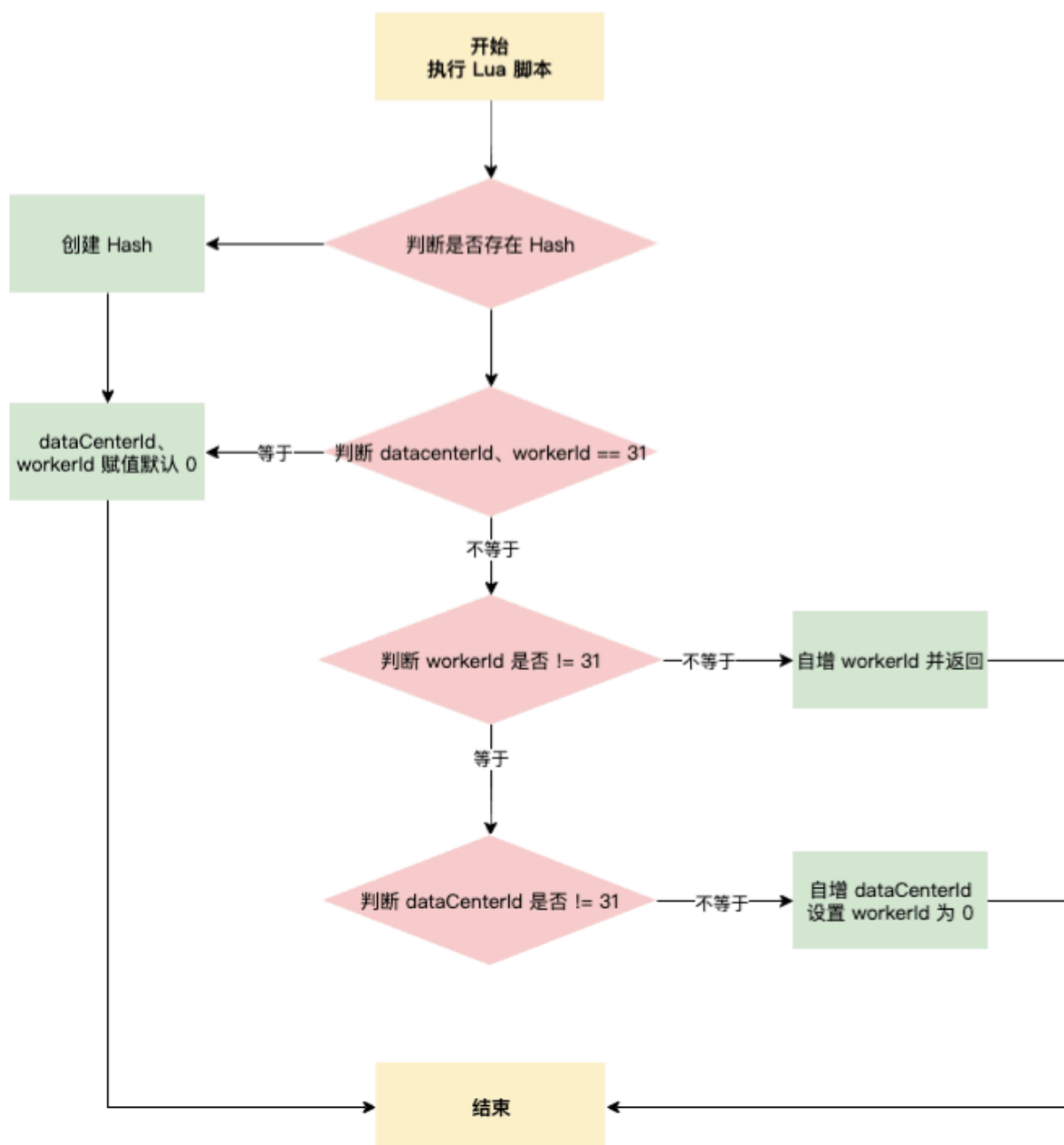
服务的节点数超过 1024，则需要做额外的扩展；可以扩展 10 bit 标识位，或者选择开源分布式 ID 框架。

动态分配实现方案

Redis 存储一个 Hash 结构 Key，包含两个键值对：dataCenterId 和 workerId。



在应用启动时，通过 Lua 脚本去 Redis 获取标识位。dataCenterId 和 workerId 的获取与自增在 Lua 脚本中完成，调用返回后就是可用的标示位。

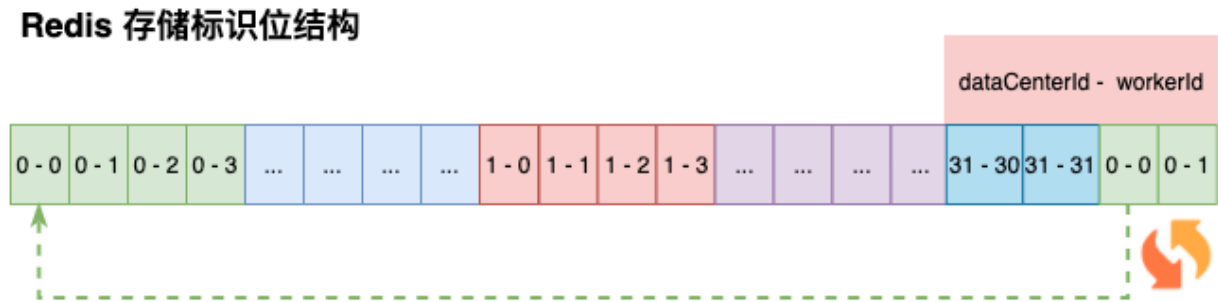


具体 Lua 脚本逻辑如下：

1. 第一个服务节点在获取时，Redis 可能是没有 snowflake_work_id_key 这个 Hash 的，应该先判断 Hash 是否存在，不存在初始化 Hash，dataCenterId、workerId 初始化为 0。
2. 如果 Hash 已存在，判断 dataCenterId、workerId 是否等于最大值 31，满足条件初始化 dataCenterId、workerId 设置为 0 返回。
3. dataCenterId 和 workerId 的排列组合一共是 1024，在进行分配时，先分配 workerId。

4. 判断 workerId 是否 $\neq 31$ ，条件成立对 workerId 自增，并返回；如果 workerId = 31，自增 dataCenterId 并将 workerId 设置为 0。

dataCenterId、workerId 是一直向下推进的，总体形成一个环状。通过 **Lua 脚本的原子性**，保证 1024 节点下的雪花算法生成不重复。如果标识位等于 1024，则从头开始继续循环推进。



开源分布式 ID 框架

Leaf 和 Uid 都有实现雪花算法，Leaf 额外提供了号段模式生成 ID。

美团 Leaf: <https://github.com/Meituan-Dianping/Leaf>

百度 Uid: <https://github.com/baidu/uid-generator>

雪花算法可以满足大部分场景，如无必要，**不建议引入开源方案增加系统复杂度。**

回顾总结

文章通过图文并茂的方式帮助读者梳理了一遍什么是雪花算法，以及如何解决雪花算法生成 ID 冲突的问题。

关于雪环算法生成 ID 冲突问题，文中给了一种方案：**分配标示位**；通过分配雪花算法的组成标示位，来达到默认 1024 节点下 ID 生成唯一。

可以去看 Hutool 或者 Mybatis-Plus 雪花算法的具体实现，帮助大家更好的理解。

雪花算法不是万能的，并不能适用于所有场景。**如果 ID 要求全局唯一并且服务节点超出 1024 节点**，可以选择修改算法本身的组成，即扩展标识位，或者选择开源方案：LEAF、UID。

更新: 2023-07-17 16:21:49

原文: <<https://www.yuque.com/magestack/12306/ciigw9ctq0v90u3w>>